Lab 1: Finding Minima of Functions

Potential energy and stable equilibrium

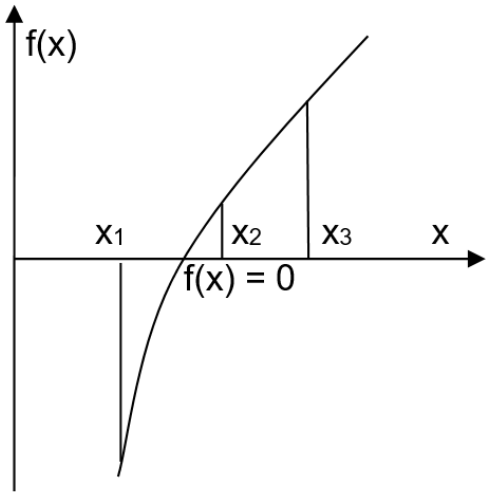
In mechanics you have met the potential energy, V(x), of a particle, which varies with the particle's position, x. A simple example is the potential $V(x) = \frac{1}{2} k x^2$ for a harmonic oscillator. Consider a particle moving along the x axis - the force on it in the x direction at the point x is the negative gradient (slope) of the potential energy function at that point,

$$F(x) = - dV(x)/dx$$

For the harmonic oscillator potential the negative gradient of the potential is $F(x) = -kx$, which is Hooke's law.

The force acting on a particle is zero at a stationary point of the function, $dV(x)/dx = 0$. Close to a potential energy minimum, the force is always towards the bottom of the potential well, while near a maximum the force is away from the top of the potential energy maximum. So the bottom of a potential well is a point of stable equilibrium, while a maximum of the potential is a point of unstable equilibrium, from which the particle tends to move away under the slightest perturbation. A point of inflection is also a point of unstable equilibrium. It is important to identify the states of stable equilibrium of particles or systems of particles, and this involves finding the minima of potential energy functions. For one-dimensional problems this means we must find the points where the slope of $V(x)$ is zero, and its second derivative is positive.

The bisection method for finding roots of a function



The bisection method for finding roots for a function, $f(x) = 0$, is very simple: start by picking an interval, bounded by x_1 and x_3, which contains a root: *i.e* $f(x_1) < 0$ and $f(x_3) > 0$.
The method searches for a root by continually decreasing the width of this interval.

Evaluate $f(x_2)$ at the midpoint of the interval:
$x_2 = (x_1 + x_3) / 2$.
If $f(x_2) > 0$, then the root lies between x_1 and x_2, so replace x_3 by x_2.
Otherwise, if $f(x_2) < 0$, replace x_1 by x_2.
Repeat (or iterate) this procedure until $|f(x_2)|$ is less than some small value (tolerance) that you choose.

Numerical accuracy in scientific computing

In mathematics there are analytical methods for finding solutions to many forms of equations. However, in many cases analytic methods are not available and we must seek numerical solutions. Numerical solutions are not, generally speaking, exact solutions to the equation concerned. Instead they are approximate solutions. For example, in the problem set here we are looking for solutions to the problem, f(x) = 0, where f is a quadratic function of x. In this particular case the analytic solution is given by

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

The numerical method we will use does not give this result exactly (unless we choose the initial guesses for the root so that we do end up coincidentally with the exact root). Instead we solve a similar problem, |f(x)| < tol, where tol is some small number which we choose.

Python Scripting

In the introductory lab, we looked at a few different options to run python code either from the command line or within a development environment such as Spyder, Jupyter Notebook or Google Colaboratory. You can try the following example using a few of these options, before choosing your preferred environment to complete the later exercises. **Remember to save all your code, output, and report files to a location which you can easily access and work from outside the lab.**

(1) Create a new file / notebook / project in the editor of your choice (e.g. Spyder / gedit in Linux / Jupyter Notebook) and ensure it is saved in an appropriate directory.

(2) A Python code, script or cell is simply a sequence of Python commands. Enter the lines below into your code, and save it (if necessary):

```
import numpy as np
import matplotlib.pylab as plt

x = np.arange(-5.0, 5.0, 0.2)
a = 1
b = 1
c = -6

plt.plot(x, a * x * x + b * x + c)
plt.plot(x, 0.0 * x)
plt.show()
```

This code uses the *arange* function, from the *numpy* library, to generate a list of values, *x*, from -5.0 to 5.0 in steps of 0.2. The two *plot()* functions plot the parabola determined by the values of a, b and c and a horizontal axis (*y=0*). Depending on the environment, the plot may not appear on screen until the *show()* command is called.

(3) Run your code,

- The green 'Run file' button in Spyder
- Running `python script.py` OR `python3 script.py` using the command line (from the directory where you saved your file as *script.py*)
- 'Shift' + 'Return' keys together inside an interactive notebook cell

Check that your code produces a plot of the parabola $f(x) = x^2 + x - 6$.

Functions in Python

When a piece of code in a script is to be used more than once, it is useful to be able to refer to a predefined function. Functions are common to most, if not all, computer languages and each language has a number of predefined (or intrinsic) functions built in. For example, sin(x), cos(x) and tan(x) are predefined and the script writer does not have to generate them. However, not all functions which are needed to solve a particular problem will be available as intrinsic functions, and it is useful to be able to define our own functions.

The syntax for a function in Python is illustrated by the example below which returns a list containing a Fibonacci series up to and including input argument, n

```
def fib(n):
    '''Return a Fibonacci series up to n'''
    a, b = 0, 1
    result=[0]
    while b < n:
        result.append(b)
        a, b = b, a+b
    return result
```

The function definition begins with `def`, the function name, `fib`, and its argument(s) in parentheses, in this case the integer, `n`. Note that the contents of the function are initially indented (by a single tab OR 4 spaces) and that the content of the *while* loop is indented by the same amount again. Indentation is very important in Python, and you will see an IndentationError message if you forget, or use the incorrect amount.

The Fibbonacci sequence is stored in a list called *result* and the line `result.append(b)` appends the latest value of *b*, generated by the while loop, to the list. If anything is to be returned to the calling part of the script, then a return statement is needed, along with the object to be returned. In this case it is the list called *result*.

To use the function to generate a list containing the Fibonacci series up to and including n = 10, it is 'called' using the code

```
fib_series = fib(10)
```

where `fib_series` is now a list of integers. Note that the function `fib` must be defined before it is called. This means the function definition (`def fib(n): …`) must appear earlier in the code than the function call (`fib_series = fib(10)`). In an interactive notebook, you must run the cell containing the definition first if it is in a different cell.

Exercise 1: Finding the roots of a parabolic function

1. Choose a parabola with a minimum and two real roots (other than the one above) Define a python function to evaluate your function for any *x*.

2. Write a python code to plot your parabola, and the line *f(x)=0*, over a suitable range. Use the function defined in step 1.

3. Add variables x_1, x_2 and x_3 to your code, with x_1 and x_3 initialised to values for which $f(x_1) < 0$ and $f(x_3) > 0$.
   Add if statements to check that these variables are initialised correctly, and warn the user if they are not.

4. Calculate the position of the midpoint $x_2 = 0.5 * (x_1 + x_3)$, and evaluate $f(x_2)$ at this point.

5. Use `if` and `else` statements to update the value of EITHER x_1 or *x3* according to whether $f(x_2)$ is greater or less than zero. Print the updated values of x_1 and x_3.

6. Plot a new point in your plot, showing $(x_2, f(x_2))$, in a different colour than the parabola curve. Make sure that the `plt.show()` command is at the end of your code, and that it remains at the end when you add further commands.

7. Initialise a variable `tol = 0.0001` to store the desired tolerance of the root-finder. Add a while loop to iteratively update x_2 using the rule $x_2 = 0.5 * (x_1 + x_3)$ while the absolute value of $f(x_2)$ is greater than the tolerance. The functions `abs()` or `np.abs()` can be used to find the absolute value.
   If your code takes more than a few seconds to run it may be stuck in an infinite loop and need to be aborted. Use Ctrl+C from the Command Line, or the 'stop' button / Interrupt Kernel in Spyder or Jupyter

8. Print the values of x_2 and $f(x_2)$ after $|f(x_2)|$ becomes less than *tol*, remembering to indent any `while`, `if` or `else` loops appropriately.
   Run your code and check that it returns a correct root.
   Modify your code so that it now finds the second root of the parabola, and run it again.

9. Add a variable *nsteps* to determine how many steps are required to find the root. Initialise this to zero outside the `while` loop, and increment it by one each time the loop executes.

10. Determine *nsteps* for different values of the tolerance *tol*.
    Plot a graph of *nsteps* versus the logarithm of *tol,* using the function `np.log10()`

You should include the graphs from steps 6 and 10 in your report.

The Newton-Raphson method

The Newton-Raphson (NR) method is based on the Taylor series expansion of a function about a specific point.

$$f(a + h) = f(a) + h\, f'(a) + h^2/2!\, f''(a) + ...$$

where $f(a)$, $f'(a)$ and $f''(a)$ are the function f and its first and second derivatives, respectively, evaluated at a. When we apply the NR method to root finding, a is the current estimate for a root and we want to know which step size and direction will take us to an improved estimate for the root. The method can be applied to find minima of a function of a single variable or to a function of many variables.

In order to have a linear equation to solve for an improved approximation for a root, the Taylor series for f about the point a is truncated at the term which is linear in h,

$$f(a + h) = f(a) + h\, f'(a) + O(h^2)$$

The term $O(h^2)$ in this equation indicates that the series was truncated and terms of order h^2 and higher were omitted. Now set $f(a+h) = 0$ so that $a + h$ is an approximation for a root. It is not the exact root since the Taylor series for f was truncated. From the truncated Taylor series, the condition $f(a + h) = 0$ is equivalent to

$$f(a) + h\, f'(a) = 0$$

from which we obtain

$$h = - f(a) / f'(a)$$

Hence, given a current estimate of the root, a, an improved estimate for the root is

$$a + h = a - f(a) / f'(a)$$

Exercise 2: Finding the roots of a function using the Newton-Raphson method

1. Create a new code which plots both your parabola from Exercise 1 and its derivative using function definitions. Note that the derivative does not depend on the constant c, so the corresponding python function only has 3 arguments (x, a, b).

2. Add a variable x_1, initialised to 1, to store the current estimate of the root.

3. Add a statement to update x_1 using the Newton-Raphson rule: $x_1 = x_1 - f(x_1) / f'(x_1)$

4. As in Exercise 1.6, plot a new point on the parabola to show $(x_1, f(x_1))$.
   Make sure that the `plt.show()` command is in the correct place so that everything required is drawn in the plot.

5. Use a *while* loop to iteratively update x_1 while $|f(x1)| > tol$, where $tol = 0.0001$.
   Be careful with indentation!

6. Print the values of x_1 and $f(x_1)$ after the absolute value of $f(x_1)$ becomes less than tol.
   Check that your code returns a correct root of the parabola.
   Modify your code to find the other root. (Think about how the algorithm works. What needs to be changed to find the other root?)

7. Use a variable *nsteps* as before to find how many steps the algorithm requires to find a root, and once more plot a graph of *nsteps* versus *tol*.
   Compare the efficiencies of the bisection and Newton-Raphson methods in your report by comparing the graphs for the two methods.

8. Experiment with the value of *tol* to find the greatest accuracy that is achievable with this method.

Ionic interaction potentials

The interaction potential between two ions such as Na^+ and Cl^- is given by

$$V(x) = A\, e^{-\frac{x}{p}} - \frac{e^2}{4\,\pi\,\varepsilon_0\,x}$$

The two terms represent short range, Pauli repulsion of electron clouds and long range electrostatic attraction of oppositely charged ions. The value of *x* which minimises *V(x)* corresponds to the equilibrium separation of the ions, i.e. the bond length.
The numerical values of $e^2/4\pi\varepsilon_o$, A and p are 1.44 eV nm, 1090 eV and 0.033 nm, respectively. Use these values in your script, as described below

Previously you used the Newton-Raphson method to find roots of functions, *f(x)*, by solving *f(x) = 0*. It can also be used to find minima or maxima of functions by solving *df/dx = 0* for *x*, since the derivative of the function vanishes at a maximum or minimum

To apply the NR method to find the root of *dV/dx = 0* we need the rule

$$x_1 = x_1 - V'(x_1) \,/\, V''(x_1)$$

where *V'* and *V''* are the first and second derivatives of *V(x)*.

Exercise 3: Finding the minima roots of a potential energy function

1. Create a new code, define a function to evaluate *V(x)* and then plot V(x) for a range of *x* values from 0.01 to 1.00.
   Use variables to store the numerical values of $e^2/4\pi\varepsilon_o$, A and p, and use units of nm and eV for length and energy, respectively.
   With the numpy library imported, you can use the function `np.exp()` to evaluate the exponential term.

2. Calculate the force acting on the particle, *F= - dV/dx* (by hand), and then create a function to evaluate it. Plot *F(x)* over the same range as *V(x)*.
   What value does *F(x)* take at the minimum of *V(x)*? Explain why in your report.

3. Differentiate *dV/dx* to obtain *V''(x)* and state what you obtain for *V'(x)* and *V''(x)* in your report.

4. Implement the Newton-Raphson method to find the value of *x* for which *V(x)* is minimum, and make sure to state it in your report.
   You should use the plot from step 1 to pick a sensible initial guess.

Supplementary Exercise: Equilibrium positions of charges on a ring

No extra marks will be awarded if you complete this exercise. It is included so you can test yourself on a harder problem and build your coding skills. The case of three inequivalent charges on a ring is an example of a problem which cannot be solved analytically, but can be solved easily using numerical methods.

Three charges of equal sign but varying magnitude are confined to move on a ring of radius, *R*. The equilibrium arrangement for the charges is one in which the potential energy function is a minimum. Since the potential energy does not change if all charges are moved on the ring through the same angle, we need to find the angles subtended by lines from the centre of the circle to the charges.

It is convenient to think of one charge as fixed on the x axis, and identify the positions of the other two by their polar coordinates (R, ϕ_1) and (R, ϕ_2). The potential energy associated with each pair is inversely proportional to the separation of the two charges. The potential energy of the 1st and 2nd charges is

$$V(r_{12}) = \frac{q_1 q_2}{4\,\pi\,\varepsilon_0 r_{12}} = \left(\frac{q_1 q_2}{4\,\pi\,\varepsilon_0}\right) \Big/ \left|2\,R\,\sin\left(\frac{\phi_1}{2}\right)\right|$$

where $r_{12} = 2R\,|\sin(\phi_1/2)|$ is the distance between the two charges, which is a positive number. Note that the separation of the 2nd and 3rd charges is $(2R)\,|\sin((\phi_2 - \phi_1)/2)|$.

1. Prove that $r_{12} = 2R\,|\sin(\phi_1/2)|$, where ϕ_1 is the angle between the charges. *Hint:* Consider formulas relating triangle side lengths to angles, together with half-angle trigonometric identities.

2. Find expressions for the force and derivative of the force between pairs of charges using the potential energy function above. Write a code to plot $V(\phi_1)$ and $F(\phi_1)$, as a function of ϕ_1, for two equal charges confined to a ring, avoiding the singularity at $\phi_1 = 0$. Explain the shape of these functions.
   (You should use the functions `np.abs()`, `np.sin()`, etc. and the variable `np.pi` from the numpy library).

3. Modify your code so that it can calculate and print i) the total energy for two charges q_1 and q_2 on a ring; ii) the forces acting on each charge. q_1 and q_2 should both be initialised to 1 for this part. The forces should be repulsive – why?

4. Use the NR method to find the equilibrium condition for three inequivalent charges. You will need nested while loops to find equilibrium configurations for all three pairs of charges.